ASGARD-XV Experiment 14

Stratospheric Balloon Atmospheric Data Logger

Hardware Design & Software Architecture
Final Description

Annáš Rychtera, Šimon Donovan, Max Dobeš, Theodor Dobeš, Jiří Dobeš

The English College in Prague

Platform: Arduino Pro Mini — ATmega328PB, 3.3 V / 8 MHz
April 2026

Table of Contents

1. Introduction

1.1 Project Objectives

1.2 Measurement Strategy

2. Hardware Design

2.1 Microcontroller Selection

2.2 Sensor Suite Overview

2.3 UV & Ambient Light Sensor — LTR-390UV-01

2.4 Barometric Pressure & Temperature Sensor — MS5607-02BA03

2.5 Electrochemical Ozone Sensor — DFRobot SEN0321

2.6 Real-Time Clock — DS3231

2.7 External EEPROM — AT24C256

2.8 MicroSD Card Module

2.9 USB-UART Programmer — CH9102

2.10 Power Supply Architecture

2.11 Bus Architecture & Wiring

3. Bill of Materials

4. Software Toolchain & Setup

4.1 Arduino IDE & Board Configuration

4.2 Installed Libraries

4.3 Compilation & Upload Procedure

5. Firmware Architecture

5.1 Initialization Sequence

5.2 Main Loop & Sensor Reading Cycle

5.3 Dual-Storage Data Recording

5.4 EEPROM Binary Record Format

5.5 MicroSD CSV Format

5.6 Serial Command Interface

5.7 Sensor Fault Tolerance & Reinitialisation

5.8 Memory Optimisation Techniques

6. Data Flow & Integration with Analysis Pipeline

7. Environmental Considerations

8. Summary

Appendix A: I2C Address Map

Appendix B: EEPROM Memory Layout

Appendix C: Sensor Flags Bitmask

Appendix D: Complete Serial Command Reference

1. Introduction

1.1 Project Objectives

This project develops an instrumented payload for a stratospheric weather balloon, designed to record a vertical profile of ozone concentration, ultraviolet irradiance, ambient light levels, atmospheric pressure, and temperature from ground level up to approximately 30–40 km altitude. The recorded in-situ data will be compared against modelled UV irradiance profiles produced by a companion Python/libRadtran radiative transfer pipeline, which ingests CAMS (Copernicus Atmosphere Monitoring Service) ozone forecast data.

The payload must satisfy the following constraints:

- **Mass:** Minimise total mass for balloon lift-budget compliance (target < 200 g for the sensor package).
- **Power:** Operate from a small battery for a 2–3 hour ascent/descent flight profile.
- **Data integrity:** Survive power loss at any point without corrupting previously recorded data; provide redundant storage.
- **Temperature range:** Function at stratospheric temperatures as low as −55 °C encountered near the tropopause (approximately 10–15 km altitude).
- **Simplicity:** Use commercially available, low-cost breakout modules with I2C/SPI interfaces; avoid custom PCB fabrication.

1.2 Measurement Strategy

The data logger records measurements at two configurable intervals:

- **MicroSD card:** Every 10 seconds (default, configurable via serial command), storing human-readable CSV files organised by date.
- **External EEPROM:** Every 30 seconds (fixed), storing compact 35-byte binary records as a resilient primary backup that survives SD card ejection, filesystem corruption, or power-bounce scenarios.

At a typical balloon ascent rate of 5 m/s, a 10-second SD interval yields one sample per 50 m of altitude — more than adequate for profiling atmospheric layers whose characteristic scales are 1–5 km.

2. Hardware Design

2.1 Microcontroller Selection

The payload is built around the **LaskaKit Arduino Pro Mini**, featuring the **Microchip ATmega328PB** microcontroller in the TQFP-32 package (datasheet). Key specifications:

Parameter	Value
MCU	ATmega328PB (superset of ATmega328P)
Operating voltage	3.3 V
Clock frequency	8 MHz (internal oscillator)
Flash memory	32 KB (0.5 KB bootloader reserved)
SRAM	2 KB
EEPROM (internal)	1 KB
I2C buses	2 (TWI0, TWI1) — PB variant advantage
UART channels	2 (UART0, UART1) — PB variant advantage
SPI	2 (SPI0, SPI1) — PB variant advantage
GPIO pins	27
ADC channels	8 (10-bit)
Operating temperature	−40 °C to +85 °C (industrial grade)

The 3.3 V variant was selected to allow direct interfacing with all sensors without level shifters. The ATmega328PB is the enhanced variant of the popular ATmega328P, providing additional hardware I2C, UART, and SPI peripherals. The industrial temperature rating of −40 °C makes it suitable for stratospheric conditions, although temperatures below this may be encountered briefly near the tropopause.

The 8 MHz clock (versus the 16 MHz typical of 5 V Arduinos) reduces power consumption and is the maximum reliable frequency at 3.3 V per the ATmega328PB datasheet speed-grade specifications.

2.2 Sensor Suite Overview

Sensor	Measurand	Interface	I2C Address	Module
LTR-390UV-01	UV index, UV raw counts, ambient light (lux)	I2C	0x53	LaskaKit LTR390 breakout
MS5607-02BA03	Barometric pressure (mbar), temperature (°C)	I2C	0x76	GY-63 compatible module
SEN0321	Ozone concentration (ppb, 0–10 ppm range)	I2C	0x73	DFRobot Gravity ozone module
DS3231	Real-time clock, on-die temperature (°C)	I2C	0x68	DS3231 + AT24C32 module
AT24C256	Non-volatile data storage (32 KB)	I2C	0x50	AT24C256 EEPROM module
MicroSD (SPI)	Bulk data storage (16 GB)	SPI	CS = D10	LaskaKit microSD module

2.3 UV & Ambient Light Sensor — LTR-390UV-01

The Lite-On LTR-390UV-01 is a dual-channel ultraviolet and ambient light sensor in a compact 2×2 mm LGA package, mounted on a LaskaKit breakout board. It contains two independent photodiodes:

- **UVS channel:** Sensitive in the 280–380 nm band (UVA/UVB), with peak sensitivity near 315 nm. This spectral range overlaps significantly with the Hartley band of ozone absorption (200–320 nm), making it directly relevant for observing the altitude-dependent attenuation of solar UV by stratospheric ozone.
- **ALS channel:** Sensitive in the visible range (approximately 500–600 nm peak), used for ambient light measurement in lux.

The sensor operates in a time-multiplexed fashion — it can measure either UV or ALS at any given moment, not both simultaneously. The firmware switches between UVS and ALS modes each cycle, with a 300 ms integration delay per mode.

Configuration Used

Parameter	Setting	Rationale
Gain	LTR390_GAIN_3 (3×)	Balanced sensitivity for both ground-level and high-altitude UV readings
Resolution	LTR390_RESOLUTION_18BIT	18-bit resolution; 100 ms nominal integration time; good noise performance
Window factor (WFAC)	1	No tinted window; sensor exposed directly to sky

The UV Index (UVI) is calculated by the library from the raw UVS counts using the formula:

```
UVI = UVS_raw / (UV_SENSITIVITY × gain_factor × res_factor × WFAC)
```

where `UV_SENSITIVITY = 1400` is the manufacturer-calibrated constant.

2.4 Barometric Pressure & Temperature Sensor — MS5607-02BA03

The MS5607-02BA03 is a high-resolution barometric pressure sensor from TE Connectivity (formerly Measurement Specialties). It is the lower-pressure-range sibling of the MS5611, optimised for altimetry applications from sea level up to very high altitudes. The module was sourced from TechFun.cz.

Parameter	Value
Pressure range	10 to 1200 mbar
Pressure resolution	0.024 mbar at OSR=4096
Temperature range	−40 to +85 °C
Temperature resolution	0.01 °C
Supply voltage	1.8 V to 3.6 V
I2C address	0x76 (CSB pin to VCC)
Oversampling	OSR_ULTRA_HIGH (4096 samples)

The MS5607's 10 mbar lower range limit corresponds to approximately 31 km altitude in the International Standard Atmosphere (ISA). Above this altitude, pressure readings will saturate at the sensor's minimum, but the payload will still record UV and ozone data. The firmware uses the `MS5607` derived class from Rob Tillaart's MS5611 library (v0.5.1), which applies the correct calibration coefficients for the MS5607 variant.

The pressure readings serve a dual purpose: (1) provide altitude estimation via the barometric formula for real-time monitoring, and (2) establish a pressure coordinate that directly maps to the CAMS ozone profile data, which uses pressure levels as its vertical coordinate.

2.5 Electrochemical Ozone Sensor — DFRobot SEN0321

The DFRobot SEN0321 Gravity Ozone Sensor is an electrochemical gas sensor module with an integrated I2C interface. It provides a direct ppb-level readout of ambient ozone concentration. Full documentation is available on the DFRobot Wiki. The module was sourced from Farnell CZ.

Parameter	Value
Measurement range	0 to 10 ppm (0 to 10,000 ppb)
Resolution	10 ppb
Response time (T90)	< 120 s
Warm-up time	~3 minutes
Operating temperature	−20 to +50 °C
Supply voltage	3.3 V to 5.5 V
I2C address	0x73 (factory default, address 3)

> **Limitation:** The SEN0321's operating temperature range of −20 to +50 °C is narrower than the stratospheric temperature profile. The sensor will likely produce unreliable or zero readings above approximately 8–12 km altitude where temperatures drop below −20 °C. This is mitigated by the sensor_flags field in the data record, which marks whether the ozone reading was valid for each sample.

The firmware uses passive measurement mode (`MEASURE_MODE_PASSIVE`) and reads averaged data over 20 samples per request (`COLLECT_NUMBER = 20`). Before each reading, an I2C presence check is performed to detect if the sensor has become disconnected or unresponsive.

2.6 Real-Time Clock — DS3231

The DS3231 is a high-accuracy, temperature-compensated real-time clock (TCXO RTC) with an integrated 32 KHz crystal oscillator, mounted on a breakout module with AT24C32 EEPROM.

Parameter	Value
Accuracy	±2 ppm (±0.17 s/day) from −40 to +85 °C
Battery backup	LIR2032 rechargeable Li-Ion (3.6 V, 40 mAh)
I2C address	0x68 (fixed)
On-die temperature sensor	±3 °C accuracy, 0.25 °C resolution

The RTC module used includes an on-board AT24C32 EEPROM (4 KB), though this is not utilised by the firmware (the separate AT24C256 provides 32 KB). The DS3231 serves three functions:

1. **Timestamping:** Provides UNIX epoch timestamps for all records, enabling correlation with CAMS forecast data and balloon GPS telemetry.
2. **SD file naming:** The current date generates daily filenames in `YYYYMMDD.CSV` format.
3. **Secondary temperature measurement:** The on-die temperature is logged alongside the MS5607 temperature for cross-validation and as a board-level thermal indicator.

If the RTC is unavailable at boot, the firmware falls back to `millis()` -based timestamps with a randomised offset (seeded from floating analog pins) to avoid timestamp collisions between power

cycles.

2.7 External EEPROM — AT24C256

A standalone AT24C256 EEPROM module (sourced from TechFun.cz) provides 32 KB (256 Kbit) of non-volatile, byte-addressable storage via I2C.

Parameter	Value
Capacity	32,768 bytes
Page size	64 bytes
Write cycle time	≤ 5 ms
Endurance	1,000,000 write cycles per page
Data retention	100 years
I2C address	0x50 (A0=A1=A2=GND)
Operating temperature	−55 to +125 °C

The EEPROM is the primary resilient storage medium. Unlike the SD card, it has no filesystem overhead, no initialisation latency, and is immune to mechanical vibration or shock that could eject an SD card. Its 8-byte header stores the current record count and next write address, which are updated after every write. The firmware respects 64-byte page boundaries during writes, chunking data into page-aligned blocks with 5 ms delays between pages.

With 35-byte records and an 8-byte header, the EEPROM holds a maximum of **935 records**. At a 30-second interval, this provides approximately **7.8 hours** of recording capacity — ample for a typical balloon flight profile of 2–3 hours.

2.8 MicroSD Card Module

The **LaskaKit microSD card module** provides SPI-based access to a standard microSD card. The module includes an on-board 3.3 V regulator and level shifters, though at 3.3 V operation the level shifters are transparent.

Parameter	Value
Interface	SPI
Chip select pin	D10
Card installed	Patriot MicroSDHC 16 GB, Class 10, UHS-I
Filesystem	FAT16/FAT32 (Arduino SD library)

The SD card serves as the secondary, high-capacity storage. It stores data in human-readable CSV format with one file per day. The 16 GB card capacity far exceeds what a balloon flight would

produce — at ~100 bytes per CSV line and one line every 10 seconds, a 3-hour flight generates approximately 108 KB of data.

2.9 USB-UART Programmer — LaskaKit CH9102

The **LaskaKit CH9102 Programmer** is a USB-to-UART bridge module with both USB-C and micro-USB connectors. It is used for:

- **Firmware upload:** Connects to the Arduino Pro Mini's UART0 (RXD0/TXD1) pins for bootloader-based programming via the Arduino IDE or `arduino-cli`.
- **Serial monitor:** Provides a 9600-baud serial interface for real-time sensor readout, command entry, and data dump.

The CH9102 includes a DTR output that, when connected to the Pro Mini's reset pin through a 100 nF capacitor, enables automatic reset for seamless upload without manual button pressing.

2.10 Power Supply Architecture

The system operates at 3.3 V. A **mini SMD step-down (buck) converter** steps down from the battery voltage to a regulated 3.3 V rail. Power is distributed as follows:

Component	Typ. Current Draw	Notes
ATmega328PB @ 8 MHz	~4 mA	Active mode
LTR-390UV-01	~0.11 mA	During measurement
MS5607	~1.4 mA	During conversion (OSR=4096)
SEN0321	~5 mA	Electrochemical cell bias
DS3231	~0.2 mA	Timekeeping active
AT24C256	~3 mA	During write, <10 µA standby
MicroSD card	~25 mA	During write burst; <0.1 mA idle
Total (peak)	**~40 mA**	All sensors + SD write

A **slide switch** (SS12D00G4, 0.3 A / 50 VDC rated) provides a physical power on/off control. The DS3231's LIR2032 rechargeable lithium coin cell (3.6 V, 40 mAh) maintains the RTC during power-off periods.

2.11 Bus Architecture & Wiring

All sensors and the EEPROM share a single I2C bus (TWI0: SDA=A4/PC4, SCL=A5/PC5). The microSD module uses the hardware SPI bus (MOSI=D11, MISO=D12, SCK=D13, CS=D10). The I2C bus topology is:

```
                      I2C Bus (SDA/SCL)
                      ═══════════════════


Arduino Pro Mini                3.3V Rail
    A4 (SDA) ───+---[4K7]───+─── VCC
    A5 (SCL) -+──┼─[4K7]---+  |
              |  |          |  |
              |  |    +─────+--+---+────────+─────────+
              |  |    |     |      |        |         |
              |  +────+--+  |  +---+--+  +--+---+  +--+---+
              +───────+  |  |  |      |  |      |  |      |
                LTR390   |  |  DS3231 |  AT24C256  EEPROM
                0x53     |  |  0x68   |  0x50   |  (on DS3231
                         |  |         |         |   module)
                     MS5607 |     SEN0321       |   AT24C32
                     0x76   |     0x73          |   0x57
                            |                   |
                            +───────────────────+

                      SPI Bus (MOSI/MISO/SCK)
                      ═══════════════════════


Arduino Pro Mini
    D11 (MOSI) ──────────→ SDI
    D12 (MISO) ←─────────── SDO      LaskaKit
    D13 (SCK)  ──────────→ CLK      microSD Module
    D10 (CS)   ──────────→ CS
```

The I2C bus uses 4.7 kΩ pull-up resistors to 3.3 V on both SDA and SCL lines. These are provided by the breakout modules (most I2C modules include on-board pull-ups), but external discrete 4.7 kΩ resistors are available for use if needed.

3. Bill of Materials

All components were sourced from LaskaKit (Czech Republic), TechFun.cz, and Farnell CZ.

3.1 Primary Components (LaskaKit)

Component	Product Code	Qty
Arduino Pro Mini, ATmega328PB, 3.3V/8MHz	LA100054A	1
LaskaKit LTR390 UV sensor breakout	LA131098	1
LaskaKit microSD card module	LA161078	1
DS3231 RTC module (with AT24C32)	LA133002	1
LaskaKit CH9102 USB-UART programmer	LA161061	1
Patriot MicroSDHC 16GB Class10 UHS-I	LA133015A	1
LIR2032 rechargeable Li-Ion battery (3.6V, 40mAh)	LA123082	1
Mini SMD step-down converter 3.3V	LA1220163	1
µŠup 4× to DIP adapter	LA150084	1
µŠup/Qwiic JST-SH 4-pin cable, 10cm	LA150124B	2
µŠup/Qwiic JST-SH 4-pin to Dupont male	LA150124D	4
µŠup-SPI JST-SH 6-pin cable, Dupont male	LA150150D	3
µŠup-SPI JST-SH 6-pin cable, Dupont female	LA150150E	3
40-conductor flat ribbon cable (26AWG), 1m	LA150154	1
Dupont M-M jumper wires 20cm (50pcs)	LA150173C	1
Dupont F-F jumper wires 20cm (50pcs)	LA150173A	1
Slide switch SS12D00G4 (0.3A 50VDC)	LA215009	2
Resistor 4.7kΩ 1% (1/4W, YAGEO MFR-25)	LA211000B	10
Resistor 100kΩ 1% (1/4W, YAGEO MFR-25)	LA211000E	10
20×80mm double-sided prototype PCB	LA150039	2
30×70mm double-sided prototype PCB	LA150011	2
Pin headers 2.54mm (various colours, 40-pin)	LA217003*	5
Socket headers 2.54mm (8/10/12-pin)	LA217005*	16
Dupont dual-row connectors (4P, 8P)	LA217030*	12
JST-SM connector set (40pcs)	LA180050	1
MicroUSB to DIP adapter	LA150027	1
DC 5.5/2.1mm to DIP adapter	LA150065	2
Maxim DS18B20+ digital temperature sensor (TO-92)	LA213000G	1

3.2 Additional Components (Other Suppliers)

Component	Supplier	Qty
MS5611-01BA03 barometric pressure sensor module	TechFun.cz	1
AT24C256 EEPROM module	TechFun.cz	1
XH2.54 6-pin connector pair (male + female)	TechFun.cz	1
M-F jumper cables 30cm (40pcs)	TechFun.cz	1
DFRobot SEN0321 Gravity Ozone Sensor	Farnell CZ	1

3.3 Datasheet Reference

Component	Manufacturer	Datasheet
ATmega328PB	Microchip	Full datasheet (PDF) | Summary (PDF)
LTR-390UV-01	Lite-On	Datasheet (PDF)
MS5607-02BA03	TE Connectivity	Datasheet (PDF)
SEN0321 Ozone Sensor	DFRobot	DFRobot Wiki | Product page
DS3231	Analog Devices (Maxim)	Datasheet (PDF)
AT24C256C	Microchip (Atmel)	Datasheet (PDF)

4. Software Toolchain & Setup

4.1 Arduino IDE & Board Configuration

The firmware is developed and uploaded using the **Arduino IDE** with the standard **Arduino AVR Boards** core (v1.8.7). Although the board carries an ATmega328PB microcontroller, the PB variant is fully backwards-compatible with the ATmega328P at the software level and works without issues under the standard Arduino board definition.

Setting	Value
Board	**Arduino Pro or Pro Mini**
Processor	**ATmega328P (3.3V, 8 MHz)**
Port	**COM5** (CH9102 USB-UART bridge)

No third-party board packages or custom board manager URLs are required. The standard Arduino AVR core, which ships with every Arduino IDE installation, provides all necessary support.

Board Configuration Steps

1. Open the Arduino IDE.
2. Go to **Tools** → **Board** and select **Arduino Pro or Pro Mini**.
3. Go to **Tools** → **Processor** and select **ATmega328P (3.3V, 8 MHz)**.
4. Go to **Tools** → **Port** and select the COM port assigned to the CH9102 programmer (typically **COM5**).

> **ATmega328PB compatibility:** The ATmega328PB is a drop-in superset of the ATmega328P. It adds extra peripherals (a second I2C, UART, and SPI bus, plus additional timers), but the base instruction set, register map, and bootloader interface are identical. The standard Arduino "ATmega328P (3.3V, 8 MHz)" target compiles and uploads correctly to the PB variant. The additional PB-only peripherals (TWI1, UART1, SPI1) are not used by this firmware.

4.2 Installed Libraries

The following Arduino libraries are installed in the user's library directory and are required to compile the firmware:

Library	Version	Author	Purpose
LTR390	1.0.9	Igor Levkov	LTR-390UV-01 UV/ALS sensor driver. Provides raw UVS/ALS readings, computed UVI and lux. Supports gain, resolution, and mode configuration.
MS5611	0.5.1	Rob Tillaart	MS5611/MS5607 barometric pressure sensor driver. Includes the `MS5607` derived class with correct calibration math for the 5607 variant. Provides temperature, pressure, and altitude calculation.
DFRobot_OzoneSensor	1.0.1	DFRobot (ZhixinLiu)	SEN0321 Gravity ozone sensor driver. Supports 4 I2C addresses, active and passive measurement modes, configurable sample averaging (1–100).
RTClib	2.1.4	Adafruit	Real-time clock library supporting DS3231, DS1307, PCF8523, and PCF8563. Provides DateTime class with UNIX timestamp conversion, alarm support, and temperature readout.
Adafruit BusIO	1.17.4	Adafruit	I2C/SPI abstraction layer. Required dependency of RTClib. Provides unified register read/write interface.
SD	(built-in)	Arduino	FAT16/FAT32 filesystem access for SD/SDHC cards over SPI. Part of the Arduino core libraries.
Wire	(built-in)	Arduino	I2C (TWI) master/slave communication. Part of the Arduino core.
SPI	(built-in)	Arduino	SPI master communication. Part of the Arduino core.

Library Installation

All non-built-in libraries can be installed via the Arduino Library Manager (**Sketch → Include Library → Manage Libraries**) by searching for their names, or manually by cloning the GitHub repositories into the Arduino libraries folder at:

```
C:\Users\<username>\Documents\Arduino\libraries\
```

4.3 Compilation & Upload Procedure

1. Open `Arduino/Arduino.ino` in the Arduino IDE.
2. Ensure the board settings are configured as described in Section 4.1.
3. Connect the CH9102 programmer to the Arduino Pro Mini:
   - VCC → VCC (3.3V)
   - GND → GND
   - TXD → RXI (Pro Mini RX, pin D0)

    - RXD → TXO (Pro Mini TX, pin D1)
    - DTR → RST (through 100nF capacitor for auto-reset)
4. Select the correct COM port under **Tools** → **Port**.
5. Click **Upload** (or press Ctrl+U).
6. Open the Serial Monitor at **9600 baud** to verify sensor initialisation messages.

5. Firmware Architecture

The entire firmware is contained in a single file: `Arduino/Arduino.ino` (773 lines). It follows a flat, procedural architecture with no state machine or task scheduler — appropriate for the ATmega328PB's limited resources and the application's straightforward sequential sensing requirements.

5.1 Initialization Sequence

The `setup()` function initialises all peripherals in the following order:

1. **Serial (UART0)** at 9600 baud — for debug output and command interface.
2. **Wire (I2C)** — initialises the TWI0 hardware peripheral.
3. **LTR390** — attempts `init()`, configures gain (3×) and resolution (18-bit), enables the sensor. Sets `ltr390Available` flag.
4. **MS5607** — attempts `begin()` on address 0x76, sets oversampling to `OSR_ULTRA_HIGH` (4096 samples). Sets `ms5607Available` flag.
5. **SEN0321** — attempts `begin(OZONE_ADDRESS_3)`, sets passive measurement mode. Sets `ozoneAvailable` flag.
6. **DS3231** — attempts `begin()`. If the RTC has lost power (backup battery depleted), automatically sets the time to the firmware compilation timestamp. If the RTC is entirely absent, generates a random time offset. Sets `rtcAvailable` flag.
7. **AT24C256 EEPROM** — reads the 8-byte header to recover the record count and next write address. Performs sanity checks; if corrupted, resets to empty state.
8. **MicroSD** — attempts `SD.begin(SD_CS_PIN)`. Sets `sdAvailable` flag.

Each sensor initialisation is independent and non-blocking. If any sensor fails to initialise, the firmware continues with the remaining sensors and marks the failed sensor's data as invalid via the `sensor_flags` bitmask. This graceful degradation ensures the payload records whatever data it can, even if individual sensors fail during the flight.

After initialisation, recording starts automatically (`recording = true`). The user does not need to send a serial command to begin data collection — critical for an unattended balloon payload.

5.2 Main Loop & Sensor Reading Cycle

The `loop()` function executes approximately once per second (dominated by the 1000 ms `delay()` at the end plus ~600 ms of sensor integration time). Each iteration performs:

1. **Serial command processing:** Reads characters from the serial buffer into a 48-byte command buffer, character-by-character. When a newline or carriage return is received, the command is parsed and executed.
2. **Timestamp acquisition:** Reads the current DateTime from the DS3231 RTC (or computes a fallback from `millis()`).
3. **UV measurement:** Switches LTR390 to UVS mode, waits 300 ms, reads raw UV counts and computed UVI.
4. **Ambient light measurement:** Switches LTR390 to ALS mode, waits 300 ms, reads raw ALS counts and computed lux.
5. **Pressure/temperature measurement:** Calls `ms5607.read()` with ultra-high oversampling; extracts temperature (°C) and pressure (mbar).
6. **Ozone measurement:** Performs an I2C presence check, then calls `readOzoneData(20)` for a 20-sample averaged reading (ppb).
7. **Live serial output:** Prints all sensor values to the serial port for real-time monitoring.
8. **Data recording:** If recording is enabled, checks whether the SD and EEPROM write intervals have elapsed and writes records as appropriate.

5.3 Dual-Storage Data Recording

The firmware employs a dual-storage strategy with independent timing:

Storage	Interval	Format	Capacity	Purpose
AT24C256 EEPROM	30 s (fixed)	35-byte binary records	935 records (~7.8 hrs)	Primary resilient backup; survives power loss, vibration, SD failure
MicroSD (FAT32)	10 s (configurable)	CSV text, daily files	16 GB (~practically unlimited)	High-resolution human-readable data; easy post-flight analysis

The SD card writes 3× more frequently than the EEPROM, providing finer altitude resolution in the primary dataset. The EEPROM provides a lower-resolution but highly reliable backup. Both timing intervals use `millis()`-based elapsed-time comparisons, which are immune to `delay()` drift.

5.4 EEPROM Binary Record Format

Each EEPROM record is exactly 35 bytes, packed as follows:

Offset	Size	Type	Field	Unit / Notes
0	4	uint32_t	timestamp	UNIX epoch seconds
4	4	uint32_t	als	Raw ambient light sensor counts
8	4	float	lux	Computed illuminance (lux)
12	4	uint32_t	uvs	Raw UV sensor counts
16	4	float	uvi	Computed UV Index
20	4	float	temperature	Degrees Celsius (MS5607)
24	4	float	pressure	Millibars (MS5607)
28	2	int16_t	ozone_ppb	Parts per billion; −1 if invalid
30	4	float	rtc_temp	Degrees Celsius (DS3231 on-die)
34	1	uint8_t	sensor_flags	Bitmask (see Appendix C)

EEPROM Memory Map

Address Range	Size	Content
0x0000 – 0x0003	4 bytes	Record count (uint32_t, little-endian)
0x0004 – 0x0007	4 bytes	Next write address (uint32_t, little-endian)
0x0008 – 0x002A	35 bytes	Record #0
0x002B – 0x004D	35 bytes	Record #1
…	…	…
0x7FE3 – 0x7FFF	35 bytes	Record #934 (last possible)

5.5 MicroSD CSV Format

Each day's data is stored in a file named `YYYYMMDD.CSV` (e.g., `20260325.CSV`). The first line is a header row, written automatically when a new day's file is created:

```
timestamp,als,lux,uvs,uvi,temp_c,pressure_mbar,ozone_ppb,rtc_temp_c,sensor_flags
```

Example data row:

```
2026/03/25 14:30:15,1234,567.89,456,3.21,22.50,1013.25,45,23.75,15
```

The filename generation uses manual character-by-character conversion to avoid `sprintf()` and its associated RAM overhead. The 8.3 filename format ensures compatibility with the Arduino SD library's FAT16/FAT32 implementation.

5.6 Serial Command Interface

The firmware implements a text-based command interface over the 9600-baud serial connection. Commands are entered one per line (terminated by `\n` or `\r`). The parser uses a fixed 48-byte character buffer with no dynamic memory allocation.

Command	Arguments	Description
`record`	[seconds]	Start recording. Optional argument sets the SD write interval (default: 10s). EEPROM interval is fixed at 30s.
`stop`	—	Stop recording. No data is written to SD or EEPROM.
`dump`	—	Dump all EEPROM records as CSV over serial. Outputs the same header and format as the SD CSV files.
`dumpsd`	[YYYY MM DD]	Dump an SD card file over serial. Without arguments, dumps today's file. With a date, dumps that day's file.
`status`	—	Display EEPROM usage (records/max, bytes/total), SD card status, current recording state and intervals.
`clear`	—	Erase all EEPROM records (resets header only; stops recording as a safety measure).
`settime`	YYYY MM DD HH MM SS	Set the DS3231 RTC to the specified date and time. Validates ranges before applying.
`help`	—	Display a summary of available commands.

String comparisons use `strcmp_P()` and `strncmp_P()` with `PSTR()` macros, which store the comparison strings in program flash memory rather than RAM — a critical optimisation on a device with only 2 KB of SRAM.

5.7 Sensor Fault Tolerance & Reinitialisation

The firmware implements a robust fault-tolerance mechanism for each sensor. A consecutive-failure counter tracks how many cycles each sensor has failed. After `SENSOR_REINIT_AFTER` (3) consecutive failures, the firmware attempts to reinitialise the sensor by repeating its `begin()` / `init()` sequence with full configuration:

- **LTR390:** Tracks UV/ALS read failures via `ltr390Fails`. On reinit, calls `init()`, `setGain()`, `setResolution()`, `enable(true)`.
- **MS5607:** Tracks read failures via `ms5607Fails`. On reinit, calls `begin()` and `setOversampling()`.
- **SEN0321:** Tracks negative readings via `ozoneFails`. Performs an I2C bus-level presence check (`Wire.endTransmission() == 0`) before each read to detect physical disconnection. On reinit, calls `begin()` and `setModes()`.
- **MicroSD:** If a write fails, immediately attempts `SD.begin()` recovery. Additionally, if the SD is unavailable, retries initialisation every 30 seconds (`SD_RETRY_INTERVAL_MS`).

The counter resets to zero on every successful read, so transient single-cycle glitches do not trigger unnecessary reinitialisations. This mechanism is particularly important for a balloon payload where sensors may experience temporary I2C bus errors due to temperature-induced timing shifts or vibration.

5.8 Memory Optimisation Techniques

With only 2 KB of SRAM available on the ATmega328PB, careful memory management is essential. The firmware employs several techniques:

1. **No Arduino `String` class:** The entire firmware avoids the Arduino `String` class, which uses dynamic heap allocation and is prone to fragmentation on small microcontrollers. All string handling uses fixed-size `char` buffers.
2. **`F()` macro for string literals:** All serial output strings are wrapped in the `F()` macro, which stores them in program flash (32 KB) instead of SRAM. This is the single most impactful RAM-saving technique, as the firmware contains dozens of diagnostic messages.
3. **`PSTR()` for comparisons:** Command string comparisons use `strcmp_P()` / `strncmp_P()` with `PSTR()` to keep comparison targets in flash.
4. **Manual number-to-string conversion:** The `getSDFilename()` function builds the 8.3 filename character-by-character using arithmetic, avoiding `sprintf()` which would pull in a large formatting library and consume ~1.5 KB of flash and stack RAM.
5. **Fixed-size buffers:** Command buffer (48 bytes), EEPROM record buffer (35 bytes), filename buffer (13 bytes), and EEPROM header buffer (8 bytes) are all statically allocated with known sizes.
6. **In-place `memcpy()` for serialisation:** EEPROM record packing and unpacking uses `memcpy()` with offset tracking rather than a struct cast, ensuring correct behaviour regardless of compiler padding and alignment settings.

6. Data Flow & Integration with Analysis Pipeline

The data logger is one component of a larger measurement-and-modelling workflow:

1. **Pre-flight:** The Python pipeline downloads CAMS ozone forecast data for the launch site coordinates (50.798°N, 4.358°E), interpolates the ozone mass mixing ratio profile to the target location, and runs libRadtran simulations to produce predicted UV irradiance at 1 km altitude intervals from 0 to 40 km.
2. **During flight:** The data logger records UV, pressure, temperature, and ozone every 10–30 seconds. The pressure readings establish an altitude coordinate that maps directly to the CAMS pressure levels.
3. **Post-flight:** The SD card CSV files (or EEPROM dumps) are retrieved and processed:
   - Pressure → altitude conversion using the ISA 1976 piecewise model.
   - UV Index profile vs. altitude plotted against the libRadtran-predicted profile.
   - In-situ ozone readings compared against the CAMS forecast profile.
   - Temperature profile compared against the ISA standard atmosphere.

The data format was designed with this pipeline integration in mind: the CSV column names match the analysis scripts' expected input, and the UNIX timestamps enable precise temporal correlation with external GPS trajectory logs provided by the ASGARD flight computer.

7. Environmental Considerations

7.1 Temperature

The stratosphere presents extreme thermal challenges. Typical temperature profiles during a balloon ascent:

Altitude	Typical Temperature	Notes
0 km (ground)	+15 °C	ISA standard
5 km	−17 °C	Lower troposphere
10–15 km (tropopause)	−55 to −60 °C	Minimum temperature
20 km	−55 °C	Lower stratosphere
30 km	−45 °C	Mid-stratosphere (warming from O3 absorption)

Component temperature ratings:

- **ATmega328PB:** −40 to +85 °C — may be marginal at the tropopause.
- **MS5607:** −40 to +85 °C — rated for the cold.
- **DS3231:** −40 to +85 °C — temperature-compensated oscillator maintains accuracy.
- **AT24C256:** −55 to +125 °C — widest range; no concerns.
- **LTR390:** −40 to +85 °C — rated for the cold.
- **SEN0321:** −20 to +50 °C — **most limiting component**. Will lose accuracy above ~8 km.

Thermal insulation (e.g., expanded polystyrene enclosure) and internal heat generation from the electronics (~130 mW) will moderate the internal temperature somewhat, but the SEN0321 ozone sensor remains the weak link. The sensor_flags field in the data allows post-flight filtering of unreliable readings.

7.2 Pressure

At 30 km altitude, ambient pressure is approximately 12 mbar (1.2 kPa). The MS5607 is rated down to 10 mbar, providing coverage up to ~31 km. Above this altitude, pressure readings will clip. The SEN0321 ozone sensor's electrochemical cell may also behave unpredictably at very low pressures, though this is secondary to its temperature limitation.

7.3 Vibration & Shock

Balloon payloads experience moderate vibration during ascent (wind gusts, pendulum oscillation) and significant shock at landing. The dual-storage strategy mitigates data loss: even if the SD card ejects on landing impact, the EEPROM retains all recorded data.

8. Summary

This document has described the design and implementation of a low-cost atmospheric data logger for stratospheric balloon profiling as part of the ASGARD-XV Experiment 14. The key design achievements are:

- **Multi-parameter sensing:** Simultaneous UV irradiance, ambient light, barometric pressure, temperature, and ozone concentration measurement from a single compact payload.
- **Robust data storage:** Dual-path recording to both EEPROM and microSD with independent intervals, ensuring data survives power loss, SD failure, or physical damage.
- **Graceful degradation:** Per-sensor initialisation flags and automatic reinitialisation mean the payload records whatever data it can, even when individual sensors fail due to extreme cold.
- **Integration-ready:** Data format directly compatible with the companion CAMS/libRadtran analysis pipeline for model-vs-observation comparison.

- **Memory-efficient firmware:** Careful avoidance of dynamic allocation, use of flash-stored strings, and manual serialisation techniques ensure reliable operation within the ATmega328PB's 2 KB SRAM constraint.

The system is designed for a typical balloon flight profile of 2–3 hours, reaching altitudes of 25–35 km, with the primary limitation being the ozone sensor's narrow operating temperature range. Future iterations could address this with a heated sensor enclosure or a different ozone sensing technology (e.g., optical absorption).

Appendix A: I2C Address Map

Address (hex)	Address (7-bit)	Device	Notes
0x50	1010000	AT24C256 EEPROM	A0=A1=A2=GND
0x53	1010011	LTR-390UV-01	Fixed address
0x57	1010111	AT24C32 (on DS3231 module)	Not used by firmware
0x68	1101000	DS3231 RTC	Fixed address
0x73	1110011	SEN0321 Ozone	OZONE_ADDRESS_3 (factory default)
0x76	1110110	MS5607 Pressure	CSB pin to VCC

All six I2C addresses are unique, so all devices can coexist on the same bus without conflict. The AT24C32 on the DS3231 module at 0x57 is present but not accessed by the firmware.

Appendix B: EEPROM Memory Layout

Region	Start Address	End Address	Size
Header	0x0000	0x0007	8 bytes
Data records	0x0008	0x7FFF	32,760 bytes

Record capacity: floor((32768 − 8) / 35) = **935 records**.

Page write behaviour: The firmware's `eepromWriteBlock()` function splits writes at 64-byte page boundaries and limits individual I2C transactions to 30 bytes (2 bytes address + 30 bytes data = 32 bytes, within the Wire library's 32-byte buffer). A 5 ms delay after each page write ensures the EEPROM's internal write cycle completes.

Appendix C: Sensor Flags Bitmask

Bit	Mask	Flag Name	Meaning when set (1)
0	0x01	FLAG_UVS_OK	LTR390 UV-channel data is valid
1	0x02	FLAG_ALS_OK	LTR390 ambient-light-channel data is valid
2	0x04	FLAG_MS5607_OK	MS5607 pressure/temperature data is valid
3	0x08	FLAG_O3_OK	SEN0321 ozone reading is valid (≥0)
4–7	0xF0	(reserved)	Unused; always 0

A `sensor_flags` value of **15 (0x0F)** indicates all four sensor channels produced valid data. Any value less than 15 means one or more sensors failed for that sample. Post-processing scripts should filter on this field.

Appendix D: Complete Serial Command Reference

Command	Syntax	Effect
record	`record [N]`	Start recording. If N is given, sets SD interval to N seconds. EEPROM interval remains 30s. Resets both interval timers.
stop	`stop`	Halt all recording. Sensors continue to be read and displayed on serial.
dump	`dump`	Output all EEPROM records as CSV (header + data rows). Includes total record count at the end.
dumpsd	`dumpsd [YYYY MM DD]`	Output the contents of an SD card CSV file. No arguments = today's file. With date arguments, dumps the specified day.
status	`status`	Display: EEPROM records used/max, bytes used/total; SD card status and today's file size; recording state and intervals.
clear	`clear`	Reset EEPROM header (sets record count and next address to 0). Stops recording as a safety measure. Does not erase SD data.
settime	`settime YYYY MM DD HH MM SS`	Set the DS3231 RTC. Validates: year 2000–2099, month 1–12, day 1–31, hour 0–23, minute 0–59, second 0–59.
help	`help`	Print a one-line summary of all commands.